ICT374 Lab 1:
Introduction to the Lab Environment; Introduction to C Programming; C's formatted output functions.

Learning Objectives:

1. Understand how the operating systems in the lab computers are set up and deployed.
2. Learn to use Linux terminal.
3. Understand the file hierarchy and the concepts of current directory, home directory, and associated commands such as `cd`, and `pwd`.
4. Learn to use basic Unix commands for file handling: `cp`, `mv`, `mkdir` and `rm`.
5. Learn to use the command `man` to view the manual pages of Unix commands.
6. Become familiar with Linux server `eris.ad.murdoch.edu.au`.
7. Understand what the four major components are in a computer.
8. Understand why the interrupt mechanism is necessary in a multi-tasking operating systemin and how an interrupt is handled by the CPU hardware and the operating system.
9. Learn to create simple C programs with a text editor such as `vi`, `emacs` or `pico`;
10. Learn to compile C programs with gcc;
11. Learn to use the standard library functions `printf`, `fgets`, `exit` etc.;
12. Learn to use standard input (`stdin`), standard output (`stdout`) and standard error (`stderr`).
13. Learn to use standard I/O functions for formatted output: `printf`, `fprintf`, and `sprintf`.

Required Reading:

- Lecture Notes for Topic 1.
- Stallongs: Chapter 1.
- LINUX: Rute User's Tutorial and Exposition (or just **Rute** for short): Chapter 4: Basic Commands.
- A mini manual for vi editor

Access to Software:

Internal students will use the lab at 460.2.044. You must bring a USB drive with you to copy your work from the lab computer. This is necessary as you cannot retain anything on any one of the lab computers.

External students should install Linux on their own computers. Before completing installation of Linux on your computers, you may temporarily use the university's Linux server `eris.ad.murdoch.edu.au` for lab work until you have successfully installed the Linux on your computer. More information on how to install Linux is available from Unit Resources page.

Lab Setup in 460.2.044

Following the following steps to login to Ubuntu Linux account "Murdoch" for this unit:

1. Boot the computer up if it is not yet booted.
2. Enter your Murdoch Username (which is your student number) and your Murdoch Password.
3. Wait for VMWare virtual machine to start up automatically.
4. Once, VMWare virtual has started, select Ubuntu from the left pane and then click the green arrow button to boot Ubuntu Linux.
5. Once Ubuntu Linux desktop is displayed, enter username and password for the shared Linux account. The username is "Murdoch" and the password is "student".
6. *Important:* the Linux was booted from a fixed image. Anything you saved in your Linux account would be completely wiped out when the Linux is booted up next time. Therefore you must copy your work from the Linux account to a separate place, such as your OneDrive, Google Drive or a USB drive when you finish your lab session. Otherwise, you will lose everything you saved in the shared Linux account.
7. *Important:* after you have finished your lab session, please log out of your Windows account and leave the computer running, or reboot the computer. Do not shutdown the computer - this is because every night at 8pm, the computer needs to run a script to update itself. If it is shutdown, it cannot update itself at night. When the computer is booted the following morning, it will have to spend a long time to update itself.

ICT374 Ubuntu Noble 202404 for Docker Desktop

ubuntu-noble-202404 is a Docker image that contains all the software you need for this unit. You can run this image on your own computer if you have Docker Desktop installed. The image is available from Docker Hub at here.

If you don't have docker Desktop yet, first install it from here.

To run the ubuntu-noble-202404 image in an AMD-64 **Windows** machine, enter the following command in a prompt:

```
docker run -it -v "[YOUR_SHARED_DIRECTORY]":/home/ubuntu --name UbuntuNoble202404 almonsalve/ubuntu-noble-202404:noble-amd64 /bin/bash
```

To run the ubuntu-noble-202404 image in a ARM-64 **M-architecture MAC**, enter the following command in the terminal:

```
docker run -it -v [YOUR_SHARED_DIRECTORY]:/home/ubuntu --name UbuntuNoble202404 almonsalve/ubuntu-noble-202404:noble-arm64 /bin/bash
```

Note that you need to replace `[YOUR_SHARED_DIRECTORY]` with the path to a directory you want to share with your Docker container on your computer.

Access to Remote Linux Servers `eris.ad.murdoch.edu.au`

The Linux server `eris` is managed by ITS and it is within the university's intranet. Every student enrolled in ICT374 is given an account on both servers.

The server runs behind Murdoch firewall. To connect to Eris from the outside of the campus, you must connect to Murdoch VPN first. Details on how to setup the VPN client on your computer is available info here and vpn.

Use command `ssh` to access the server. This command is available on any Linux, macOS and Windows 10. Start up a terminal on Linux/macOS, or Command Prompt or PowerShell if you use Windows 10, and then type the following command:

```
ssh your_user_name@eris.ad.murdoch.edu.au
```

You can transfer files between your local computer and the remote `eris` server using command `sftp`. This command is available on any Linux, macOS and Windows 10. Start up a terminal on Linux/macOS, or Command Prompt or PowerShell if you use Windows 10, and then type the following command:

```
sftp your_user_name@eris.ad.murdoch.edu.au
```

More details on how to use `sftp` are available in Lab 3.

Unix Exercises

The following exercises are designed to get familiar with Unix (Linux) systems. More information about these basic Unix commands is available from the Rute: Chapter 4 (see Required Reading).

1. Log in to your Linux account.
2. Play with the Linux desktop environment for while.
3. Start up a terminal. Try the command `ls -l` to see what is under your home directory.
4. Use manual page command to check various options available to the ls command: `man ls`.
5. Learn to use command `cd` to move around in the directory hierarchy. For example, type `cd /bin` and then `ls -l` to see a list of commonly used commands. To return to your home directory, type `cd`.
6. Use command `pwd` to find out your current directory. Experimenting with the commands `cd`, `pwd` and `ls`.
7. Try to use a text editor to create a text file. I recommend `vi` editor (see the mini manual for vi under Required Reading), however you may use any other text editor such as `emacs`, `pico`, and `nano`.
8. Use the command `cat` and `less` to view the contents of a text file.
9. Use command `mkdir` to create a directory.
10. Experiment with the commands `cp`, `mv`, `rm` to copy, move and remove files around the directory hierarchy. Check their manual pages if you do not know how to use these commands (eg, `man cp`).
11. Use command `mkdir` to create a new directory and use command `rmdir` to remove any empty directory.
12. Zip a directory on eris server: From time to time, we need to create a zip archive for a directory so that we can move the content in the directory to somewhere else (e.g., transfer the content from eris to your PC, or submit a lab to the LMS). This is easily achieved using command `zip` on Unix systems. For example, to archive directory `lab01` and all its contents, use the following command:

    ```
    zip -r lab01.zip lab01
    ```

    To unzip the file, use command:

    ```
    unzip lab01.zip
    ```

Theory Exercises

1. What are the four main components in a computer?

2. We know that an operating system is a program. Explain the main difference between an operating system and other non-operating system programs.

3. What is an interrupt? Why the PC register and PSW register must be saved before anything else when processing an interrupt? What saving these two registers must be done by the hardware rather than software?

4. The operating system on your computer system shares the processor (assuming there is only one processor) with many other programs. However, we understand that at any given moment, the processor can only execute the instructions from one program. Explain how can the operating system and several other programs run on the same processor at the same time.

C Programming Exercises

Note, all exercises must be done via the terminal commands, not GUI tools. Do not use any IDE to build your programs. Do not use GUI file managers to manage your directories and files. Also each exercise should be in a separate directory, which itself is under a directory called `lab01`

1. Use command `mkdir` to create a new directory named `lab01` under your home directory:

    ```
    %   cd
    %   mkdir lab01
    ```

    In the above example, the symbol "%" denotes your shell prompt. Your actual shell prompt may vary. Shell prompt can be configured to whatever you like. The command `cd` sets your current directory to your home directory.

    Now enter the directory `lab01` and then create a subdirectory named `c_ex1`:

    ```
    %   cd lab01
    %   mkdir c_ex1
    ```

    Use the commands `cd`, `pwd` and `ls -l` to confirm that the above directory hierarchy has been created.

Now go to the directory `~/lab01/c_ex1` using the command

```
%   cd ~/lab01/c_ex1
```

In the above command, the symbol "~" stands for the home directory of the current user. If a user' username is `john`, then `~john` stands for user `john`'s home directory. If a student's username is his student number, such as `12345678`, then this student's home directory is `~12345678`.

Now use a text editor, such as `vi` or `pico`, but not GUI based text editors, to create a C program hello.c that outputs `hello world`.

Name the file `hello.c` and save it in directory `c_ex1`.

Compile your program with command `gcc hello.c`. This command compiles the source code `hello.c` into an object code `hello.o`, and then links the object code with the necessary library routines to form a complete, executable, program named `a.out`.

Now list your current directory with command `ls -lt`. You will see your original source code file `hello.c`, possibly the object code `hello.o` and the executable `a.out`.

Now run the program `a.out` by typing the command `./a.out`. Note in this command, the dot indicates that the program `a.out` is located under the current directory (a single dot "." represents the current directory and double dots ".." represents the parent directory). You should see the output `Hello world!`.

In C, we use functions such as `printf`, `putchar`, etc., to send output to the standard output, which is similar to `System.out.print` in Java and `cout <<` in C++. To use these functions, we must include header file `<stdio.h>` (standard I/O header file).

To see how to use the functions such as `printf`, type the manual command: `man -s 3 printf` on the terminal. In this command, the option "-s 3" means section 3 of the manual. All standard C functions are grouped under section 3.

Try to view the manual pages for the standard C functions `printf` and `exit`.

2. Create a C source code getinput.c that prompts the user to enter his full name. Save it in an appropriately named file such as `getFullName.c`. Compile it and execute it.

   Note the above program is part of C Exercise 2 in Lab 1, therefore, it should be placed inside directory `lab01/c_ex2`.

   Some of you may realize that the program name `a.out` isn't always a good choice. If you wants to use a different name for your executable such as `getFullName`, use the option `-o` of gcc, as in `gcc getFullName.c -o getFullName`. The option "-o" stands for "Output".

   Recompile your program to create an executable named `getFullName`.

   Note that, by convention, Unix commands or programs do not use extension names, unlike Windows (which uses `.exe` and `.com`). Therefore please do not use extension names for your Unix executables.

   In the above program, we use the standard I/O function `fgets` to read a line of input from a file, which in this case is the standard input (`stdin`). Both `fgets` and `stdin` are defined in the standard I/O library, therefore, you must include header file `<stdio.h>` in your program.

   `stdin` (standard input) is similar to `System.in` in Java, and `cin` in C++. `stdout` (standard output) is similar to `System.out` in Java, and `cout` in C++. `stderr` (standard error) is similar to `System.err` in Java, and `cerr` in C++.

   Note also in the above program, we read a line into a character array of size 128 characters. If you end the keyboard input with a ENTER key, `fgets` will store the newline character in the character array. In some applications, you may need to strip this character off from the char array. For example the following code can be used to strip the newline character away from the char array `line`:

   ```
   n = strlen(line);
   if (line[n-1] == '\n')
       line[n-1] = '\0';
   ```

   In C, the last character in a string must always be the null character "\0".

3. Write a C program that reads a line of input from the standard input and then reverse the line. Finally print out the reversed line on the standard output.

   You may want to use the standard string function `strlen` to find out the number of characters in a character array. By typing the command `man -s 3 strlen`, you will see that you must include `string.h` header file.

   You may also want to use C's for-loop for this question. The following example shows how to loop through an array:

   ```
   n = strlen(line);
   for ( i = 0; i < n; ++i ) {
       c = line[i];
   }
   ```

   where `n` is an integer variable containing the number of characters in the char array `line`, `i` is an integer variable which acts as the loop index, `c` is a char variable, and `line` is a character array.

4. **Formatted Output with `printf`**

   In the previous exercise, we used the `printf` function to send output to the standard output. This function allows you to construct complex strings consisting of strings, integers, floats, chars, etc, in a very flexible way. For example, the following code constructs the string

   ```
   Today is 0008 August 2020.
   ```

   from integer variable `day`, string variable `month` and integer `year`:

   ```
   int day = 8;
   ```

```
char *month = "August";
int year = 2020;

printf ("Today is %04d %s %d.\n", day, month, year);
```

In the above `printf` function, each "%" starts a conversion specification. For example, the first conversion `%04d` means converting the first variable, `day`, to a decimal string using at least 4 digits with zero padding. The second conversion `%s` means convert the second variable, `month`, to a string. The third conversion, `%d`, means converting the third variable, `year`, into a decimal string without padding.

Convert the above code into a complete C program. Run the program to send output to your terminal screen.

Use standard output redirection, so that the program sends its output to a file named `foo`.

Don't forget to place the files for this exercise inside subdirectory `lab02/c_ex1`.

5. **Sending Output to a File with `fprintf`**

   What if you want to send output to a specific file, such as `foo2`, rather than to the standard output in your program? Well, you can use another standard function similar to `printf` : called `fprintf`. But first you must open file `foo2` to get a file stream:

   ```
   char *fname = "foo2";
   FILE  *fstr;

   fstr = fopen(fname, "w+");
   if ( fstr == NULL ) {
       fprintf(stderr, "Cannot open file %s, terminate program\n", fname);
       exit(1);
   }

   fprintf(fstr, "bla bla bla ... to %s\n", fname);
   ```

   In the above code, `fopen` returns a pointer to `FILE` if the file is opened for writing successfully by the program code. Otherwise it returns `NULL`.

   If `fopen` fails, we send an error message

   ```
   Cannot open file foo2, terminate program
   ```

   to the standard error. Note that you do not need to open standard error. It is always open automatically.

   Finally we can send something to file `foo2` using `fprintf`.

   Note `printf(.....)` is equivalent to `fprintf(stdout, .....)`.

   For further details about function `fprintf`, type the command:

   ```
   man  -s 3  fprintf
   ```

   Modify the program from Exercise 1 so that the output is sent to file `foo2` directly in the program (rather than using standard output redirection).

   Please close the file once you have finished using it by

   ```
   fclose(fstr);
   ```

6. **Generate Formatted String with `sprintf`**

   Sometimes we want to contruct a complex string to be used within the program rather than to send to the user. This can be done using another standard function similar to `printf` and `fprintf` called `sprintf`. For example, the following code sends the constructed string to the char array `long_date`:

   ```
   #define BUF_SIZE  256

   int day = 8;
   char *month = "August";
   int year = 2006;
   char long_date[BUF_SIZE];

   sprintf (long_date, "Today is %04d %s %d.\n", day, month, year);
   ```

   Modify Exercise 1 to send the constructed string to a char array. Then print the array to the standard output.

The Weekly Lab

Submit the following exercises:

1. Theory Exercise 3
2. C Programming Exercise 2
3. C Programming Exercise 3
4. C Programming Exercise 5

This weekly lab must be submitted to the LMS. The deadline of submission is given in the Unit Information Page of the LMS. You must adhere to the instructions in the document titled The Requirements for Documenting and Submitting Weekly Labs when preparing and submitting a weekly lab.

Final Reminder:

Please remember to copy your work files to your USB drive.

Dr Hong Xie
Unit Coordinator

Last update: Tuesday, 29 July 2025

Document author: Dr. H. Xie, Unit Coordinator, ICT374 Operating Systems and Systems Programming
Last Modified: Monday, 28-Jul-2025 Alvaro Monsalve
Disclaimer & Copyright Notice © 2025 Murdoch University
This document is relevant for 2025 only